

Aula 03 — Objetos, referências e identidade

O que exatamente é copiado quando uma variável de objeto é atribuída a outra?

PONTO DE PARTIDA

No Laboratório 02...

- criamos objetos com `new` ;
- atribuímos estado;
- chamamos comportamentos com `.` ;
- usamos variáveis como `item1` e `item2` .

Uma linha conhecida

```
ItemPedido item1 = new ItemPedido();  
item1.quantidade = 2;
```

`new ItemPedido()` cria um objeto.

UMA ATRIBUIÇÃO

Agora aparece outra variável

```
ItemPedido item2 = item1;
```

O que essa instrução fez?

ATIVIDADE

UMA ATRIBUIÇÃO

Preveja antes de executar

1. Quantos objetos existem?
2. Os dados foram copiados para um novo objeto?
3. Alterar `item2.quantidade` afeta o valor observado por `item1` ?

O EXPERIMENTO

Vamos observar

```
ItemPedido item1 = new ItemPedido();  
item1.quantidade = 2;  
  
ItemPedido item2 = item1;  
item2.quantidade = 7;  
  
System.out.println(item1.quantidade);
```

O resultado

CONSOLE

7

A alteração foi feita usando `item2` e observada usando `item1`.

O EXPERIMENTO

Se houve uma cópia independente...

Por que `item1.quantidade` também passou a mostrar `7` ?

Três partes, papéis diferentes

```
ItemPedido item1 = new ItemPedido();
```

ItemPedido

tipo da variável

item1

variável

A variável é o objeto?

A variável e o objeto têm papéis diferentes no programa.

- o objeto possui estado e comportamento;
- a variável permite chegar até ele.

INVESTIGANDO A LINHA

Precisamos nomear essa relação

```
item1 —> objeto ItemPedido
```

O que a variável mantém para permitir o acesso ao objeto?

CONCEITO-CHAVE

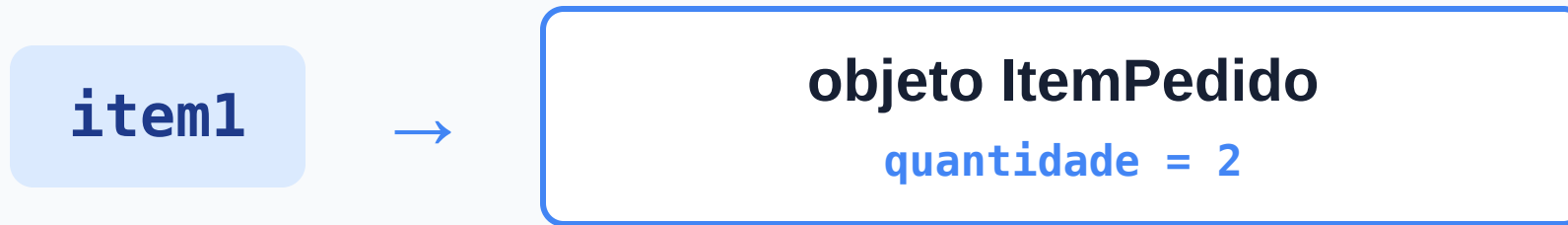
O RESULTADO OBSERVADO

Referência

Um valor que permite localizar e acessar um objeto. Uma variável de tipo de classe pode manter esse valor.

REFERÊNCIA

Uma variável, um acesso



REFERÊNCIA

O que a atribuição copia?

```
ItemPedido item2 = item1;
```

O valor de `item1` é uma referência. É esse valor que a atribuição copia.

REFERÊNCIA

Duas variáveis, um objeto



Agora o resultado faz sentido

```
item2.quantidade = 7;  
System.out.println(item1.quantidade); // 7
```

A alteração ocorreu no único objeto acessado pelas duas variáveis.

ARMADILHA

INTERPRETAÇÃO TENTADORA

Outra variável não significa outro objeto

`ItemPedido item2 = item1;` não executa `new` .

A instrução copia a referência; não cria uma cópia independente do objeto.

QUANTOS OBJETOS?

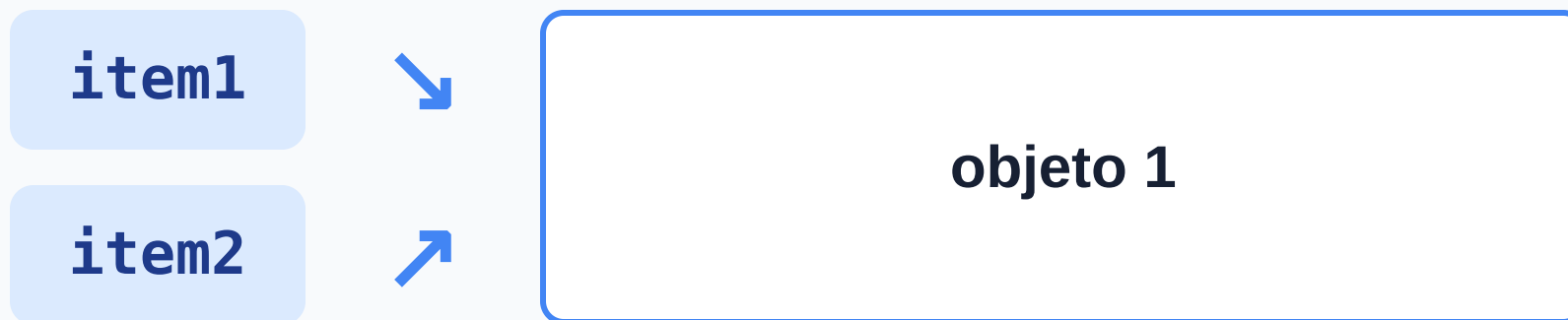
Cenário A — uma criação

```
ItemPedido item1 = new ItemPedido();  
ItemPedido item2 = item1;
```

Quantas variáveis? Quantos objetos?

QUANTOS OBJETOS?

Cenário A — duas referências



Duas variáveis. Um objeto.

QUANTOS OBJETOS?

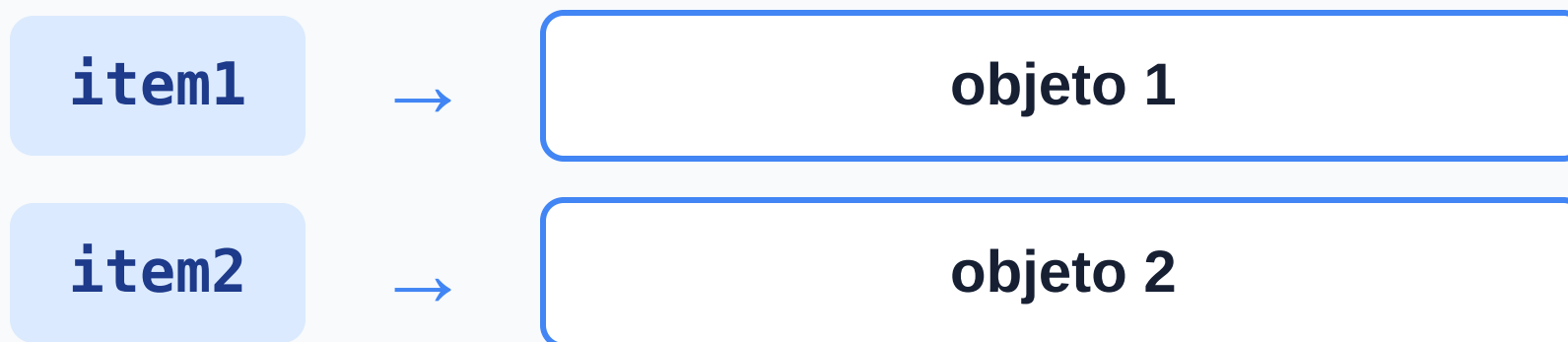
Cenário B — duas criações

```
ItemPedido item1 = new ItemPedido();  
ItemPedido item2 = new ItemPedido();
```

Quantas variáveis? Quantos objetos?

QUANTOS OBJETOS?

Cenário B — objetos distintos



Duas variáveis. Dois objetos.

QUANTOS OBJETOS?

A pista está na criação

Cada execução de `new ItemPedido()` cria um novo objeto.

Contar variáveis e contar objetos são tarefas diferentes.

E se os valores forem iguais?

```
ItemPedido item1 = new ItemPedido();  
item1.descricao = "Teclado";  
item1.quantidade = 2;
```

```
ItemPedido item2 = new ItemPedido();  
item2.descricao = "Teclado";  
item2.quantidade = 2;
```

OBJETOS DISTINTOS

Estados equivalentes

objeto 1

descricao = "Teclado"
quantidade = 2

objeto 2

descricao = "Teclado"
quantidade = 2

Os valores observados são iguais.

Duas perguntas diferentes

1. Os objetos possuem os mesmos valores?
2. As variáveis permitem acesso ao mesmo objeto?

Estado responde à primeira pergunta. Ainda precisamos nomear a segunda.

CONCEITO-CHAVE

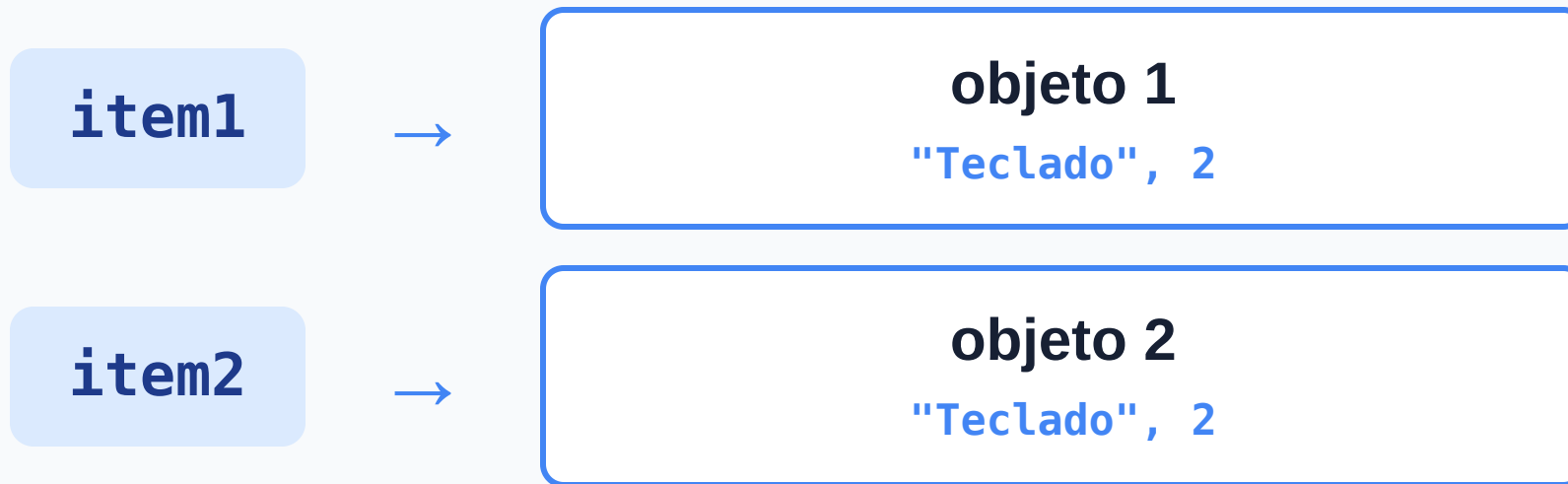
DOIS OBJETOS

Identidade

Distingue um objeto dos demais. Objetos criados separadamente continuam distintos, mesmo quando possuem o mesmo estado.

IDENTIDADE

Mesmo estado, identidades diferentes



IDENTIDADE

Como verificar se é o mesmo objeto?

Até aqui, usamos o diagrama e contamos execuções de `new` .

Qual mecanismo Java responde diretamente a essa pergunta?

Primeiro cenário

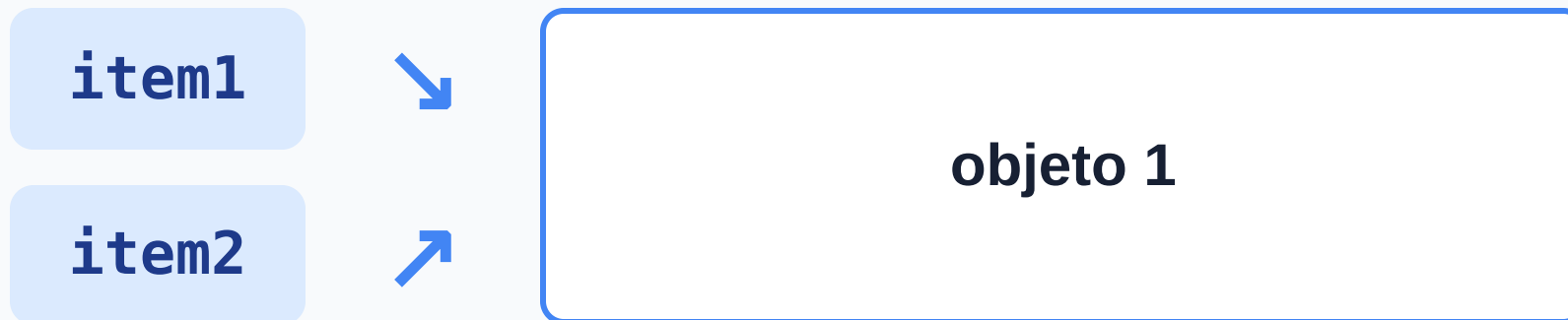
```
ItemPedido item1 = new ItemPedido();  
ItemPedido item2 = item1;  
  
System.out.println(item1 == item2);
```

Qual resultado esperamos?

IDENTIDADE

O mesmo objeto

true



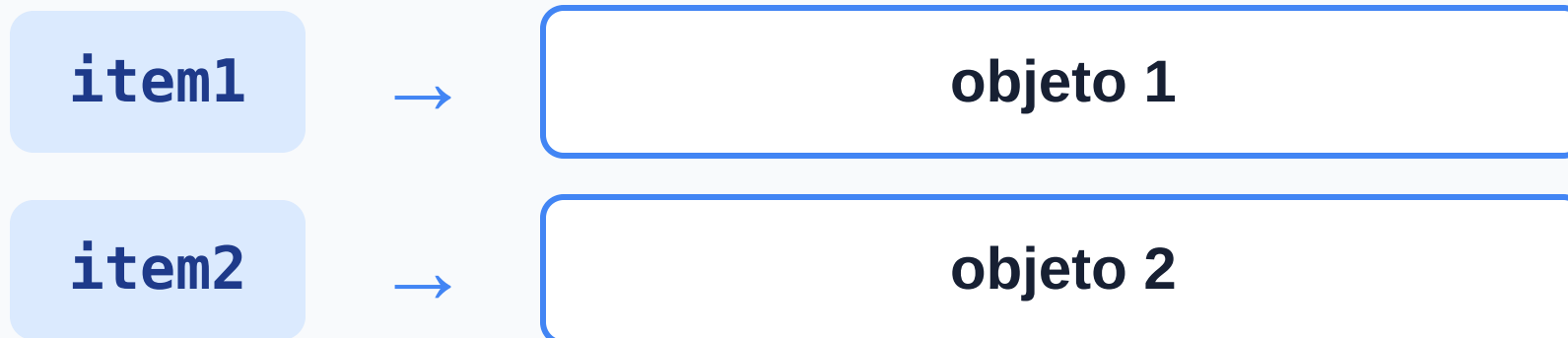
Segundo cenário

```
ItemPedido item1 = new ItemPedido();  
ItemPedido item2 = new ItemPedido();  
  
System.out.println(item1 == item2);
```

Qual resultado esperamos?

Objetos distintos

false



== entre referências

`item1 == item2` verifica se as duas referências correspondem ao mesmo objeto.

- mesmo objeto → `true` ;
- objetos distintos → `false` .

Reúna as pistas

```
ItemPedido a = new ItemPedido();  
a.quantidade = 2;  
  
ItemPedido b = a;  
  
ItemPedido c = new ItemPedido();  
c.quantidade = 2;  
  
b.quantidade = 4;
```

Desenhe, preveja e justifique

1. Quantos objetos existem?
2. Quais variáveis acessam o mesmo objeto?
3. Quais quantidades serão observadas por `a`, `b` e `c`?
4. Quanto produzem `a == b` e `a == c`?

O diagrama explica o código



As previsões se conectam

Estado observado

`a.quantidade` → 4

`b.quantidade` → 4

`c.quantidade` → 2

Identidade

`a == b` → true

`a == c` → false

TRANSFERÊNCIA

O domínio mudou. O modelo não.

```
Conta contaPrincipal = new Conta();  
Conta contaParaConsulta = contaPrincipal;
```

Se uma operação feita por `contaParaConsulta` alterar o objeto, o que será observado por `contaPrincipal` ?

Da atribuição à identidade

`new` cria



variável mantém referência



atribuição copia referência



identidade distingue



`==` verifica

PRÓXIMO PASSO

Se duas partes acessam o mesmo objeto...

```
item.quantidade = -200;
```

Qualquer uma deveria poder alterar diretamente seu estado?